

Two Pointers — Complete Guide

1. What is it?

Two Pointers is a technique where you use **two index variables** (pointers) to traverse a data structure (usually an array, string, or linked list) instead of nested loops, to reduce time complexity from $O(n^2)$ to $O(n)$ or $O(n \log n)$.

The pointers can move:

- **Toward each other** (from both ends, converging)
- **In the same direction** (fast/slow, or a sliding window)
- **From different starting points** (merging two structures)

2. Why is it used?

- Avoids brute-force nested loops ($O(n^2) \rightarrow O(n)$)
- Constant extra space — no hash maps needed in most cases
- Naturally exploits **sorted order** or **monotonic property** in data
- Elegant for problems involving pairs, subarrays, or in-place modification

3. Where is it used?

- Sorted array pair/triplet sum problems
- Removing duplicates / elements in-place
- Palindrome checking
- Merging sorted arrays/lists
- Sliding window (subarray/substring) problems
- Container/trapping water problems
- Linked list cycle detection (Floyd's — fast/slow pointers)
- Partitioning (Dutch National Flag, quicksort partition)

4. How to identify a Two Pointers problem (pattern triggers)

Ask these questions within the first 2 minutes:

Signal in the problem statement	What it suggests
"sorted array" / "sorted linked list"	Converging or same-direction pointers
"pair/triplet that sums to X"	Opposite-direction pointers
"in-place", "O(1) space"	Fast/slow same-direction pointers
"subarray/substring with property (sum, distinct chars, at most K...)"	Sliding window (variable two pointers)
"remove duplicates", "move zeroes"	Slow/fast write-read pointers
"palindrome"	Converging pointers from ends
"merge two sorted..."	Two pointers, one per structure
"cycle in linked list", "middle of linked list"	Fast/slow (Floyd's)
"container", "trapping rain water", "max area"	Converging pointers, greedy shrink
Asks for $O(n)$ or $O(n \log n)$ but brute force is $O(n^2)$ or $O(n^3)$	Strong hint to look for two pointers

Paraphrase test: If the problem can be rephrased as *"find two indices i, j such that some condition holds"* or *"find the longest/shortest contiguous range where condition holds"* — it's very likely two pointers.

5. Can you tell from constraints alone?

Yes, constraints are a strong secondary signal:

- n up to $10^5 - 10^6$ with expected $O(n)$ or $O(n \log n)$ → nested loops are out, two pointers/sliding window is a prime candidate.
- Array is stated or can be sorted without breaking the answer (order doesn't matter, e.g., "does a pair exist") → sort + converging pointers.

- If order **matters** and you can't sort (e.g., original indices needed) → use a hashmap instead, not two pointers.
- "At most/exactly/at least K distinct" with large n → sliding window (variant of two pointers).

6. Intuition / Rationale

Nested loops recheck combinations blindly. Two pointers work because of a **monotonic invariant**: moving one pointer only ever helps in one direction, so you never need to re-examine a discarded state.

Example: in sorted array pair-sum, if `arr[left] + arr[right] > target`, decreasing `right` is the *only* useful move — increasing `left` while keeping `right` fixed only makes the sum bigger or same relative issue. This monotonicity is what lets you drop one loop entirely — each pointer moves at most n times total, giving O(n).

7. Time & Space Complexity (generic)

- **Time**: O(n) or O(n log n) if sorting is required first
- **Space**: O(1) extra (excluding input/output), since only index variables are used

8. Types of Two Pointers (with templates)

Type A — Opposite Direction (Converging)

Used for: sorted array pair sum, palindrome check, container with most water, trapping rain water, reverse array in place.

```
public boolean twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) {
            return true; // or record indices/pair
        } else if (sum < target) {
            left++; // need a bigger sum
        } else {
            right--; // need a smaller sum
        }
    }
    return false;
}
```

```
}
```

Type B — Same Direction (Fast/Slow, in-place read-write)

Used for: remove duplicates, move zeroes, partition array.

```
public int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;
    int slow = 0; // last position of "clean" section
    for (int fast = 1; fast < arr.length; fast++) {
        if (arr[fast] != arr[slow]) {
            slow++;
            arr[slow] = arr[fast];
        }
    }
    return slow + 1; // new length
}
```

Type C — Sliding Window (Variable-size, same direction)

Used for: longest/shortest subarray or substring satisfying a condition.

```
public int longestSubarrayWithSumAtMostK(int[] arr, int k) {
    int left = 0, sum = 0, maxLen = 0;
    for (int right = 0; right < arr.length; right++) {
        sum += arr[right];
        while (sum > k) { // shrink window while invalid
            sum -= arr[left];
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Type D — Fast/Slow Pointers (Cycle detection — Floyd's)

Used for: linked list cycle detection, finding the middle/duplicate.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
}
```

```

        if (slow == fast) return true;
    }
    return false;
}

```

Type E — Merge Pointers (Two separate structures)

Used for: merge two sorted arrays/lists, intersection of sorted arrays.

```

public int[] mergeSorted(int[] a, int[] b) {
    int i = 0, j = 0, k = 0;
    int[] result = new int[a.length + b.length];
    while (i < a.length && j < b.length) {
        result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];
    return result;
}

```

9. Comparison of the patterns

Type	Pointer movement	Typical trigger words	Complexity
A. Opposite/Converging	left++ or right– based on comparison	"sorted", "pair sum", "palindrome", "container"	O(n)
B. Fast/Slow (in- place)	both move forward, different speeds/roles	"remove", "in-place", "move zeroes"	O(n)
C. Sliding Window	right always advances, left conditionally	"subarray", "substring", "at most/exactly K"	O(n)
D. Floyd's Cycle	fast moves 2x speed of slow	"cycle", "linked list", "middle node"	O(n)
E. Merge	independent pointers on two structures	"merge", "two sorted arrays"	O(n+m)

Key distinguishing question: *Are there two separate collections (→ Merge), one collection with a fixed-size shrink (→ Converging), one collection with a growable/shrinkable range (→ Sliding Window), or a "rewrite in place" (→ Fast/Slow)?*

10. Quick decision checklist (use this in an interview)

1. Is data sorted or sortable without losing needed info? → **Converging**
2. Am I looking for a contiguous range satisfying a condition? → **Sliding Window**
3. Am I overwriting an array in place / removing elements? → **Fast/Slow read-write**
4. Am I merging or comparing two separate sorted structures? → **Merge pointers**
5. Is it a linked list with cycle/middle-finding needs? → **Floyd's fast/slow**

If none of these fit and elements need to be looked up regardless of order/position, it's probably a **hashmap** problem instead, not two pointers.